

Open in app ↗

Sign up

Sign in

Medium

 Search Write

Encoder vs. Decoder in Transformers: Unpacking the Differences



Hassaan Idrees

Follow

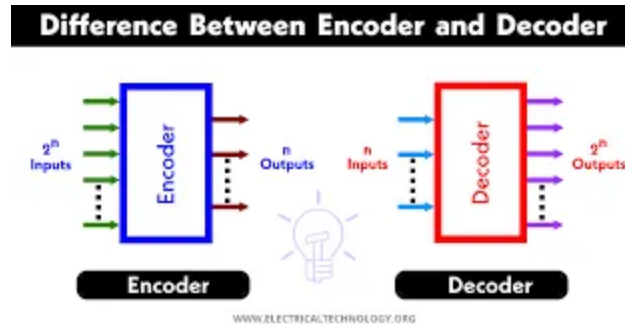
3 min read · Jul 29, 2024



Understanding the Core Components of Transformer Models and Their Roles

Introduction

In the realm of Transformers, two key components stand out: the encoder and the decoder. Understanding the roles and differences between these components is essential for students and practitioners working with Transformer models in natural language processing (NLP) and other fields. This blog post delves into the concepts, functions, and applications of encoders and decoders in Transformers, providing a clear and comprehensive comparison.



What is an Encoder?

The encoder is a fundamental component of the Transformer model, responsible for processing input sequences and generating context-aware representations.

Key Concepts:

- **Input Processing:** The encoder takes the input sequence and transforms it into a series of high-dimensional vectors.
- **Self-Attention Mechanism:** Utilizes self-attention to capture relationships between all tokens in the input sequence.
- **Layer Stacking:** Typically consists of multiple layers, each containing a self-attention mechanism and a feed-forward neural network.

Mathematical Formulation: For each input token, the encoder computes:

$$A(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

Function:

- **Contextual Representation:** Generates context-aware representations for each token in the input sequence.
- **Position Awareness:** Incorporates positional encoding to maintain the order of tokens.

What is a Decoder?

The decoder is the counterpart to the encoder, designed to generate output sequences from the encoded representations.

Get Hassaan Idrees's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

Key Concepts:

- **Output Generation:** Takes the encoded input representation and generates the output sequence, one token at a time.
- **Masked Self-Attention:** Uses masked self-attention to ensure that the generation of each token depends only on previously generated tokens, not future ones.
- **Encoder-Decoder Attention:** Incorporates attention over the encoder's output to integrate information from the input sequence.

Mathematical Formulation: For each output token, the decoder computes:

$$\text{MaskedAttention}(Q, K, V) = \text{softmax} \left(\frac{QK^T + M}{\sqrt{d_k}} \right) V$$

Function:

- **Sequential Output:** Generates the output sequence in an autoregressive manner.
- **Integration of Input Context:** Utilizes encoder-decoder attention to integrate contextual information from the encoder's output.

Key Differences Between Encoder and Decoder

Role:

- **Encoder:** Processes and encodes the input sequence into context-aware representations.
- **Decoder:** Generates the output sequence from the encoded representations, ensuring each token is generated based on previous tokens.

Attention Mechanism:

- **Encoder:** Uses self-attention to capture relationships within the input sequence.
- **Decoder:** Uses masked self-attention to ensure causal dependency in output generation and encoder-decoder attention to incorporate input context.

Application:

- **Encoder:** Suitable for tasks that require understanding and encoding input sequences, such as text classification and sentiment analysis.
- **Decoder:** Essential for tasks that involve generating sequences, such as machine translation and text generation.

Applications of Encoder and Decoder

Encoder-Only Models:

- **BERT (Bidirectional Encoder Representations from Transformers):** Utilizes the encoder to produce deep contextual representations for tasks like question answering and text classification.

Decoder-Only Models:

- **GPT (Generative Pre-trained Transformer):** Uses the decoder to generate coherent and contextually relevant text based on input prompts.

Encoder-Decoder Models:

- **Transformers for Machine Translation:** Models like the original Transformer and T5 (Text-to-Text Transfer Transformer) leverage both encoder and decoder for translating text between languages.

Conclusion

The encoder and decoder are integral components of Transformer models, each playing distinct roles in processing and generating sequences. Understanding their functions and differences is crucial for effectively applying Transformer models to various NLP tasks. By mastering these

components, students and practitioners can leverage the full potential of Transformers in their projects. Share your thoughts and questions in the comments below, and stay tuned for more insights into the world of machine learning and NLP.

AI

Artificial Intelligence

Encoder Decoder

**Written by Hassaan Idrees**

168 followers · 1 following

Follow

Hassaan is an aspiring Data Scientist with a passion for self-directed learning in ML, eager to showcase his proficiency in NLP, CV, and various ML techniques.

Responses (1)



Write a response

What are your thoughts?

**Shanyiwen A Student**

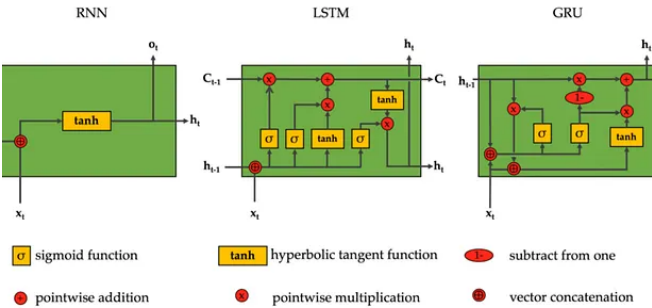
Feb 13, 2025



Tasteless blog generated by AI.

 1 [Reply](#)

More from Hassaan Idrees



 Hassaan Idrees

RNN vs. LSTM vs. GRU: A Comprehensive Guide to...

Introduction

Jul 4, 2024

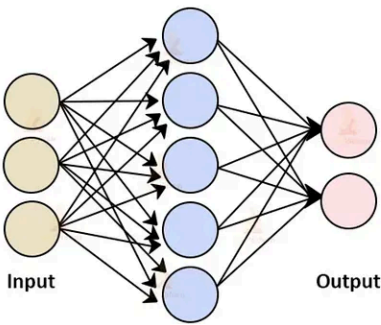
 164

 1



	Features	
Global Context	Captures global relationships between patches in a single layer	Requires deep layers to build global understanding from local features
Image Processing	Divides the image into non-overlapping patches	Scans the image using overlapping receptive fields (filters)
Positional Encoding	Requires explicit positional encoding for patch location awareness	Implicitly encodes position through convolution structure
Model Complexity	Higher computational cost due to attention mechanism	Typically less computationally expensive than ViTs
Data Requirements	Requires large datasets to perform well	Performs well on both small and large datasets
Scalability	Highly scalable with larger datasets and bigger models	Scales well, but prone to overfitting on small datasets
Inductive Biases	Minimal inductive bias; learns relationships directly from data	Strong inductive bias toward local features (edges, textures)
Performance on Small		Perform better on smaller datasets due to

 Hassaan Idrees



 Hassaan Idrees

ANN vs. CNN vs. RNN vs. LSTM: Understanding the Differences in...

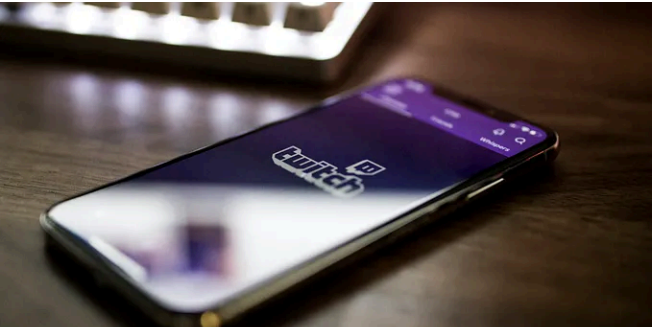
Introduction

Jul 3, 2024

 29

 6





 Hassaan Idrees

Vision Transformer vs. CNN: A Comparison of Two Image...

Understanding the Key Differences Between Vision Transformers (ViT) and Convolutional...

Sep 6, 2024



21



3



Autoencoders vs. PCA: Dimensionality Reduction for...

A Comparative Analysis of Neural Networks and Principal Component Analysis for Data...

Oct 23, 2024

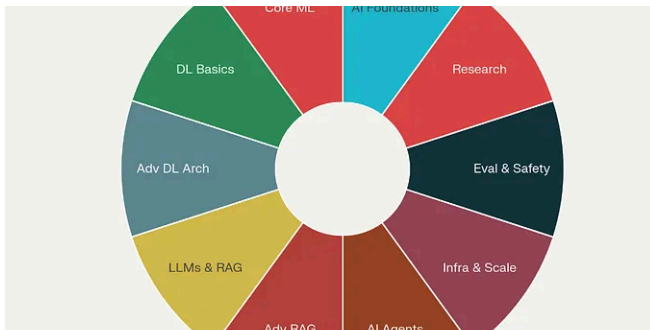


10



See all from Hassaan Idrees

Recommended from Medium



Kiran vutukuri

Stop Chasing Tools:The Engineer's Guide to AI From Models to...

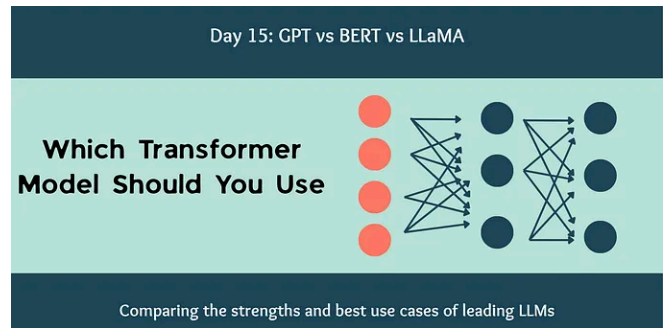
Most people jump straight to tools or models and get stuck when things break. This...



Dec 26, 2025



161



Maheera Amjad

GPT vs. BERT vs. LLaMA: Which Transformer Model Should You...

Comparing the strengths and best use cases of leading LLMs



Oct 24, 2025



61





 In Intuitively and Exhaustively Ex... by Daniel Warf...

Positional Encoding—Intuitively and Exhaustively Explained

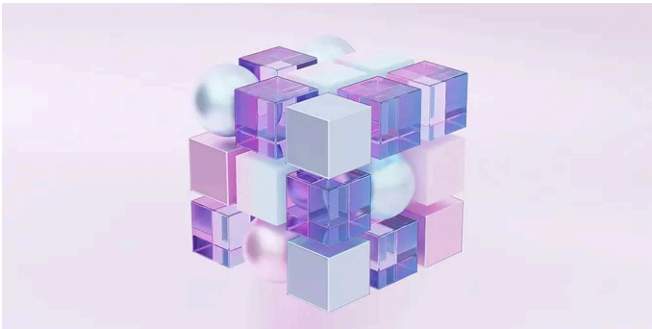
How modern AI understands space and time

★ Aug 5, 2025

👤 357

💬 6

🔖



 In Towards AI by Kuriko Iwai

The Transformer Model—A Deep Dive into Core Mechanisms

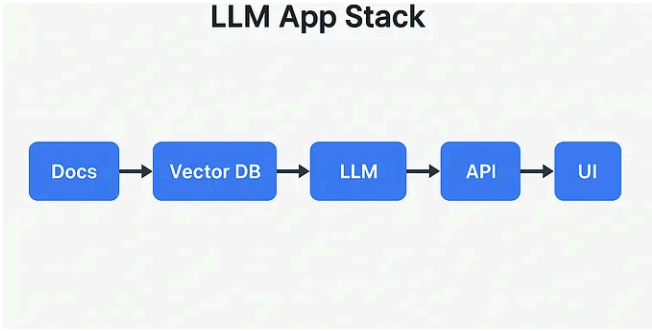
Exploring attention and its role in contextual text understanding with walkthrough...

★ Sep 9, 2025

👤 549

💬 4

🔖



 Brian Curry

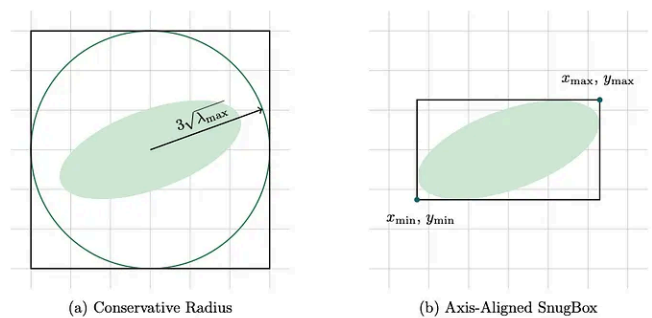
How to Build and Deploy an LLM in One Hour

You don't need a research lab, GPUs, or weeks of effort. With the right tools, you can stand ...

★ Sep 13, 2025

👤 1

🔖



 Papers in 100 Lines of Code

Accelerating 3D Gaussian Splatting With Speedy Splat

3D Gaussian Splatting (3DGS) is one of the fastest rendering methods in neural graphic...

★ Dec 18, 2025

👤 1

🔖

See more recommendations